

Evaluación Sumativa #2 (25%): Aplicación web con Django Admin.

ÁREA ACADÉMICA	Área Informática, Ciberseguridad y Telecomunicaciones	CARRERA	Ingeniería en informática / Analista Programador
ASIGNATURA	Programación Back End (TI3041)		
SEDE	La Serena	DOCENTE	<i>Alex Díaz Araos</i>
DURACIÓN		FECHA	
A. Esperado:	2.1 Codifica aplicaciones web para proponer soluciones tentativas a una problemática, utilizando un framework del lado del servidor y herramientas de inteligencia artificial como apoyo al desarrollo.		

NOMBRE ESTUDIANTE:			
	Apellido Paterno	Apellido Materno	Nombres
RUT:		-	
PUNTAJE MÁXIMO	100	NOTA:	Firma conforme
PUNTAJE OBTENIDO			
Solicita re-corrección	Sí No	Motivo:	

Situación de Evaluación: Evolución del Proyecto Desarrollado en la Evaluación Sumativa N°1

Contexto

En la evaluación anterior se desarrolló un sitio web modular utilizando Django Framework, compuesto por dos aplicaciones independientes que permitían visualizar información almacenada en archivos JSON.

La organización requiere ahora ampliar dicha solución, incorporando persistencia de datos mediante una base de datos relacional, administración de información a través de Django Admin y funcionalidades CRUD completas para la gestión de la información.

Adicionalmente, para simular un escenario real de producción, la solución deberá ser desplegada y ejecutada en una instancia Amazon EC2 de AWS, utilizando GitHub como mecanismo de control de versiones y distribución del proyecto.

El objetivo es transformar el prototipo desarrollado en la Evaluación Sumativa N°1 en una aplicación web funcional basada en base de datos, administrable y desplegada en infraestructura cloud.

Instrucciones

De forma individual, continúe el desarrollo del proyecto realizado en la Evaluación Sumativa N°1, manteniendo las dos aplicaciones implementadas y migrando la gestión de información desde archivos JSON hacia una base de datos relacional.

La solución deberá estar completamente operativa al momento de la revisión presencial.

Requerimientos de Infraestructura

AWS EC2

El proyecto deberá ejecutarse dentro de una instancia EC2 de AWS.

La instancia deberá contener:

- Sistema operativo Linux.
- Python instalado.
- Entorno virtual operativo.
- Django Framework instalado.
- Servidor web ejecutando la aplicación.
- Git instalado.
- Base de datos operativa.

Durante la revisión el estudiante deberá:

- Conectarse a su instancia EC2.
- Ejecutar el proyecto desde la instancia.
- Demostrar el funcionamiento de la aplicación

Requerimientos de Control de Versiones

Git y GitHub

Todo el proyecto deberá mantenerse bajo control de versiones.

Se debe demostrar:

- Existencia de un repositorio GitHub propio.
- Historial de commits asociados al desarrollo.
- Configuración del repositorio remoto.
- Clonación del proyecto desde GitHub hacia la instancia EC2 mediante comandos Git.

Evidencias mínimas

Durante la revisión presencial el estudiante deberá mostrar:

- Repositorio remoto.
- Historial de commits.

Y demostrar que el proyecto fue obtenido mediante: **git clone URL_DEL_REPOSITORIO**

Requerimientos de Base de Datos

Persistencia de Información

La información anteriormente almacenada en archivos JSON deberá migrarse completamente hacia una base de datos relacional.

La estructura de datos deberá corresponder al modelo definido por el estudiante para su proyecto, considerando todas las entidades necesarias para resolver la problemática planteada.

No se establece una cantidad mínima de tablas; sin embargo, deberán implementarse todas las tablas contempladas en el modelo de datos diseñado para la solución.

Modelado con Django ORM

Se deberá utilizar:

- Modelos Django.
- Relaciones entre entidades.
- Llaves foráneas (ForeignKey) cuando corresponda.
- Migraciones.
- Consultas ORM.

Evidencias mínimas

Durante la revisión presencial el estudiante deberá demostrar los **archivos de migraciones realizados**.

y explicar:

- Las entidades creadas.
- Las relaciones existentes.
- La finalidad de cada tabla dentro de la solución.

Requerimientos de Variables de Entorno

Las configuraciones sensibles NO podrán quedar escritas directamente en el código.

Se deberá utilizar un archivo con la información sensible, cargado mediante librerías apropiadas.

Verificación de Base de Datos

Toda la estructura de base de datos deberá encontrarse creada en la instancia EC2.

Durante la revisión presencial el estudiante deberá demostrar:

Desde Django

- Modelos implementados.
- Migraciones ejecutadas.
- Registros creados mediante Django Admin.

Desde phpMyAdmin

- Existencia física de todas las tablas.
- Estructura de las tablas.

- Relaciones implementadas.
- Registros almacenados.

El docente verificará la consistencia entre:

- Modelos Django.
- Migraciones.
- Base de datos.
- Información observada en phpMyAdmin.

Requerimientos de Django Admin

Todas las entidades definidas en el modelo deberán estar registradas y operativas en Django Admin.

No se aceptarán entidades que existan en los modelos pero que no puedan administrarse desde Django Admin.

Funcionalidades mínimas requeridas

Desde Django Admin deberá ser posible:

- Crear registros.
- Modificar registros.
- Eliminar registros.
- Visualizar registros.
- Buscar registros.
- Navegar entre entidades relacionadas.

La revisión de las funcionalidades CRUD se realizará exclusivamente a través del panel de administración de Django.

Requerimientos Funcionales de la Aplicación

La aplicación web deberá permitir visualizar la información almacenada en la base de datos mediante plantillas Django.

Todas las vistas deberán obtener la información mediante consultas realizadas utilizando Django ORM.

Visualización de Datos

Cada módulo del sistema deberá contar con vistas que permitan mostrar los registros almacenados en la base de datos.

Ejemplos:

- Listado de productos.
- Listado de categorías.
- Listado de clientes.
- Listado de reservas.
- Listado de videojuegos.
- Listado de películas.

o cualquier otra temática definida por el estudiante.

La información deberá presentarse mediante:

- Tablas HTML.
- Tarjetas Bootstrap.
- Listas estructuradas.

Navegación

La interfaz deberá permitir acceder a los distintos módulos del sistema mediante enlaces de navegación implementados con Bootstrap.

Botones de Acción

Cada módulo deberá incorporar visualmente los controles que serán utilizados en la siguiente evaluación para implementar operaciones CRUD desde la interfaz de usuario.

Por lo tanto, cada vista de listado deberá incluir los siguientes elementos:

- Botón Agregar.
- Botón Modificar.
- Botón Eliminar.
- Botón Buscar.

Estos controles:

- ✓ Deben existir visualmente en la interfaz.
- ✓ Deben mantener una estructura coherente con Bootstrap.

- ✓ Deben enlazar a una ruta o marcador de posición.
- ✗ No es requisito que ejecuten acciones reales.
- ✗ No es requisito que permitan modificar datos.
- ✗ No es requisito que se encuentren operativos.

La implementación funcional de estas operaciones será evaluada en la siguiente evaluación sumativa.

Revisión Presencial Obligatoria

Durante la revisión el estudiante deberá demostrar:

Infraestructura

- ✓ Conexión a EC2
- ✓ Proyecto clonado desde GitHub
- ✓ Entorno virtual activo

Base de Datos

- ✓ Todos los modelos creados
- ✓ Migraciones aplicadas
- ✓ Tablas visibles en phpMyAdmin
- ✓ Registros almacenados

Django Admin

- ✓ Administración de todas las entidades
- ✓ Creación de registros
- ✓ Edición de registros
- ✓ Eliminación de registros
- ✓ Búsqueda de registros

Front End

- ✓ Listados contruidos desde datos almacenados en la base de datos
- ✓ Uso de Django ORM
- ✓ Tablas o vistas de visualización funcionando

- ✓ Botones Agregar, Modificar, Eliminar y Buscar visibles en la interfaz

Control de Versiones

- ✓ Repositorio GitHub
- ✓ Historial de commits
- ✓ Clonación del proyecto desde GitHub hacia EC2

Entregables

1. Proyecto Django desplegado en AWS

La aplicación deberá encontrarse:

- Ejecutándose desde la instancia EC2.
- Con base de datos completamente funcional.
- Con operaciones CRUD operativas.
- Con administrador Django operativo.

2. Repositorio GitHub

Debe contener:

- Código fuente completo.
- Historial de desarrollo.
- Archivo README.
- Archivo .gitignore adecuado.

3. Documento Técnico (PDF o Word)

Debe incluir:

Descripción del proyecto

- Objetivo.
- Temática elegida.
- Funcionalidades implementadas.

Arquitectura

- Estructura de carpetas.
- Aplicaciones desarrolladas.
- Base de datos utilizada.

Evidencia AWS

Capturas de:

- Instancia EC2.
- Terminal Linux.
- Ejecución del proyecto.

Evidencia GitHub

Capturas de:

- Repositorio.
- Commits.
- Clonación del proyecto.

Evidencia Base de Datos

Capturas de:

- Modelos Django.
- Migraciones.
- Tablas creadas.

Evidencia phpMyAdmin

El estudiante deberá demostrar mediante capturas:

- Existencia de las tablas.
- Registros almacenados.
- Relaciones implementadas.

Evidencia de IA

- Prompts utilizados.
- Respuestas obtenidas.
- Aplicación de las respuestas en el desarrollo.

Escala de Apreciación

Criterio de Evaluación	Puntaje Máximo	L	ML	PL	NL	Puntaje Obtenido
Configura correctamente una instancia AWS EC2 y despliega la aplicación Django demostrando su funcionamiento durante la revisión.	15	15	11	6	0	15
Utiliza Git y GitHub para el control de versiones, evidenciando repositorio remoto, historial de commits y clonación hacia AWS mediante Git.	10	10	7	4	0	10
Implementa variables de entorno para la gestión de configuraciones sensibles de la aplicación y conexión a base de datos.	10	10	7	4	0	10
Diseña e implementa correctamente los modelos de datos mediante Django ORM, considerando entidades, relaciones y migraciones funcionales.	20	20	14	8	0	20
Administra todas las entidades del sistema mediante Django Admin, permitiendo crear, modificar, eliminar, visualizar y buscar registros.	15	15	11	6	0	15
Visualiza correctamente los datos almacenados en la base de datos mediante templates Django, utilizando consultas ORM y presentando la información en tablas o componentes equivalentes.	20	20	14	8	0	20
Presenta evidencias del uso de herramientas de Inteligencia Artificial durante el desarrollo del proyecto.	5	5	4	2	0	5
Entrega documentación técnica completa y organizada con las evidencias solicitadas.	5	5	4	2	0	5
TOTAL	100	Total puntaje obtenido				100
		Nota				7

Nivel	Descripción	Factor
Logrado (L)	Cumple completamente el criterio solicitado.	100%
Medianamente Logrado (ML)	Cumple parcialmente el criterio, presentando errores	70%
Por Lograr (PL)	Cumple de forma insuficiente el criterio,	40%
No Logrado (NL)	No presenta la evidencia o esta no funciona.	0%

Escala de Notas:

Puntaje	Nota	Puntaje	Nota	Puntaje	Nota	Puntaje	Nota	Puntaje	Nota	Puntaje	Nota	Puntaje	Nota	Puntaje	Nota
0	1,0	13	1,7	26	2,3	39	3,0	52	3,6	65	4,4	78	5,4	91	6,3
1	1,1	14	1,7	27	2,4	40	3,0	53	3,7	66	4,5	79	5,4	92	6,4
2	1,1	15	1,8	28	2,4	41	3,1	54	3,7	67	4,5	80	5,5	93	6,5
3	1,2	16	1,8	29	2,5	42	3,1	55	3,8	68	4,6	81	5,6	94	6,6
4	1,2	17	1,9	30	2,5	43	3,2	56	3,8	69	4,7	82	5,7	95	6,6
5	1,3	18	1,9	31	2,6	44	3,2	57	3,9	70	4,8	83	5,7	96	6,7
6	1,3	19	2,0	32	2,6	45	3,3	58	3,9	71	4,8	84	5,8	97	6,8
7	1,4	20	2,0	33	2,7	46	3,3	59	4,0	72	4,9	85	5,9	98	6,9
8	1,4	21	2,1	34	2,7	47	3,4	60	4,0	73	5,0	86	6,0	99	6,9
9	1,5	22	2,1	35	2,8	48	3,4	61	4,1	74	5,1	87	6,0	100	7,0
10	1,5	23	2,2	36	2,8	49	3,5	62	4,2	75	5,1	88	6,1		
11	1,6	24	2,2	37	2,9	50	3,5	63	4,2	76	5,2	89	6,2		
12	1,6	25	2,3	38	2,9	51	3,6	64	4,3	77	5,3	90	6,3		